

Technical Requirements Document (TRD)

Nearby Wells Intelligence System (NWIS) — Technical Blueprint & Architecture

Client / Organization: Oil India Limited (OIL)

Initiative: Smart India Hackathon (SIH 2026)

Document Type: Technical Requirements Document (TRD)

Version: 1.0 (Production Blueprint)

Classification: Engineering Specification

Technical Requirements Document (TRD)

Nearby Wells Intelligence System (NWIS)

Oil India Limited — SIH 2026

Version: 1.0

Status: SIH Prototype / Technical Blueprint

1. System Overview

NWIS is a modular geospatial + data engineering + ML + RAG + real-time analytics platform.

It has two major technical stages:

1. Offline knowledge construction

- Ingest historical well data and reports.
- Extract, normalize and structure drilling knowledge.
- Build spatial, depth and event indexes.
- Train/evaluate ML components.

2. Online intelligence

- Query nearby wells for a proposed location.
- Correlate depth, geology and historical events.
- Generate evidence-backed risk insights.
- Later ingest live eRTMAC/WITSML data for early warning.

2. High-Level Architecture



3. Complete Backend Pipeline

The backend should be implemented as an orchestrated pipeline.

```

INPUT
↓
VALIDATE
↓
SPATIAL RETRIEVAL
↓
WELL PROFILE RETRIEVAL
↓
DOCUMENT / LOG RETRIEVAL
↓
DEPTH NORMALIZATION
↓
EVENT EXTRACTION
↓
CROSS-WELL CORRELATION
↓
WELL SIMILARITY
↓
RISK INFERENCE
↓
HISTORICAL EVIDENCE RETRIEVAL
↓
RECOMMENDATION
↓
API RESPONSE
↓
DASHBOARD

```

Backend Processing Stages

Stage	Processing	Output
1	Location intake	Proposed well object
2	PostGIS spatial search	Nearby wells
3	Well metadata retrieval	Well profiles
4	Document intelligence	Searchable document chunks
5	Depth normalization	Comparable depth index
6	Event extraction	Structured drilling events
7	Cross-well correlation	Historical patterns
8	Similarity model	Comparable well ranking
9	Risk model	Depth-wise risk
10	RAG retrieval	Evidence package
11	Recommendation	Preventive suggestions
12	API orchestration	Dashboard JSON
13	Live monitoring	Early-warning alerts

4. Data Acquisition Strategy

4.1 Open Data for SIH Prototype

There is no assumed single public dataset that perfectly matches Oil India WCR + DDR + mud logging + eRTMAC + historical event requirements.

Therefore the prototype should combine multiple public/open sources.

Equinor Volve Open Data

Useful for:

- Well logs
- Mud logs
- Reports
- Technical well data
- Daily Drilling Reports
- WITSML real-time drilling data

Use in NWIS:

- Historical well intelligence
- Drilling event timelines
- Drilling-parameter features
- Real-time stream replay

NLOG Netherlands

Useful for:

- Public boreholes
- Reports/logs
- LAS/LIS digital logs
- Samples
- Cores
- Lithostratigraphy
- Deviation data

Use in NWIS:

- Geospatial well discovery
- Geological similarity
- Well logs
- Formation information
- Trajectory information

NLOG PSNS

Useful for:

- Pressure-related well data
- Completion reports
- Formation tests
- DST
- FIT/LOT
- Mud reports/logs
- Drilling reports
- Well logs

Use in NWIS:

- Pressure context
 - Historical drilling indicators
-

5. Production / Enterprise Data

When authorized Oil India data is available, the same schema should support:

- WCRs
- DDRs
- Mud logging records
- Historical drilling reports
- Well headers
- Well coordinates
- Trajectories
- Formation tops
- Geological data
- Casing/cementing programs
- Mud programs
- BHA/bit information
- Historical drilling events
- Reservoir data
- eRTMAC/WITSML/API data

The architecture should not need to be rebuilt; only the connectors, mappings and training data need to be expanded/retrained.

6. Data Ingestion Pipeline

```
RAW FILES / API
↓
LANDING STORAGE
↓
SCHEMA VALIDATION
↓
FILE PARSING
↓
OCR (if required)
↓
UNIT NORMALIZATION
↓
DEPTH NORMALIZATION
↓
EVENT EXTRACTION
↓
FEATURE ENGINEERING
↓
TRAINING / QUERY TABLES
↓
POSTGRES + POSTGIS + PGVECTOR
```

Supported Input Types

- PDF
- HTML
- XML
- CSV
- JSON
- LAS
- LIS
- WITSML

Important rule

Original source documents should remain immutable and versioned.

7. Document Intelligence Pipeline

```
PDF / REPORT
↓
TEXT EXTRACTION
↓
OCR IF SCANNED
↓
SECTION DETECTION
↓
CHUNKING
↓
ENTITY EXTRACTION
↓
EVENT EXTRACTION
↓
DEPTH / FORMATION LINKING
↓
SOURCE EVIDENCE STORAGE
```

Entities can include:

- Well
- Depth
- Formation
- Event
- Severity
- Drilling parameter
- Mitigation action
- Outcome
- Date

Every extracted event should retain:

- Source document
- Page
- Evidence text
- Extraction method
- Confidence

8. Data Normalization

Units

Normalize:

- meters / feet
- bar / psi
- SG / ppg

- Celsius / Fahrenheit

Depth References

Maintain:

- MD
- TVD
- TVDSS

Do not silently convert all depths into one number without retaining the original reference.

Event Taxonomy

Create controlled event classes such as:

- Mud Loss
- Kick
- Stuck Pipe
- Fishing
- Pressure Anomaly
- Torque/Drag Anomaly
- Casing Issue
- Cementing Issue
- NPT

Aliases should be mapped to a common event type.

9. Database Architecture

Recommended Core

- PostgreSQL
- PostGIS
- pgvector
- S3-compatible object storage / MinIO

Core Tables

Table	Important Fields
wells	well_id, name, operator, latitude, longitude, dates, total_depth
well_trajectory	well_id, MD, TVD, TVDSS, inclination, azimuth
formations	well_id, formation, top_depth, base_depth, lithology
well_logs	well_id, depth, curve_name, value, unit, quality_flag
drilling_parameters	well_id, timestamp, depth, WOB, RPM, torque, ROP, SPP, flow, hookload, mud_weight
events	event_id, well_id, event_type, start_depth, end_depth, severity, source, confidence
event_evidence	event_id, document_id, page, evidence_text, extraction_confidence
documents	document_id, well_id, type, source_uri, checksum, date
mitigations	event_id, action, outcome, source_document, confidence
well_similarity	source_well_id, target_well_id, similarity_score, feature_version
risk_predictions	plan_id, depth_start, depth_end, event_type, probability, confidence, model_version
alerts	alert_id, well/plan, depth, event, severity, evidence, status

10. Spatial Search Pipeline

Input

- Proposed latitude
- Proposed longitude
- Search radius

Query Concept

Use PostGIS:

```
SELECT
  well_id,
  name,
  ST_Distance(
    geography(ST_MakePoint(longitude, latitude)),
    geography(ST_MakePoint(:lon, :lat))
  ) AS distance_m
FROM wells
WHERE ST_DWithin(
  geography(ST_MakePoint(longitude, latitude)),
  geography(ST_MakePoint(:lon, :lat)),
  :radius_m
)
ORDER BY distance_m;
```

The backend can additionally calculate:

- distance
- bearing
- direction
- field/block
- drilling year

11. ML Architecture

Use specialized modules.

Module	Purpose	Baseline
Well similarity	Comparable well retrieval	Feature similarity + ranking
Event classifier	Event classification	XGBoost / LightGBM / Random Forest
Depth-risk model	Event probability	Gradient boosting + calibration
Anomaly detector	Live anomaly detection	Isolation Forest / robust statistics
RAG retriever	Historical evidence	Embeddings + pgvector/Qdrant
AI assistant	Natural-language answer	Approved LLM/local model

12. Feature Engineering

Spatial Features

- Distance
- Bearing
- Field/block membership

Geological Features

- Formation
- Lithology
- Porosity/permeability
- Resistivity
- Gamma ray
- Other available log curves

Drilling Features

- Depth
- ROP

- WOB
- RPM
- Torque
- SPP
- Flow
- Hookload
- Mud weight
- ECD where available

Historical Features

- Event type
- Event depth
- Event severity
- NPT
- Mitigation outcome

Trajectory Features

- Inclination
- Azimuth
- Dogleg severity

13. Training Dataset Construction

Example training record:

Field	Example
well_id	VOLVE-15
depth_start	2750
depth_end	2800
formation	Formation-X
mud_weight	1.18 SG
ROP	18 m/h
WOB	120 kN
torque	12 kNm
SPP	180 bar
event_type	mud_loss
event_label	1
severity	moderate
source_type	DDR / report / derived
source_id	DOC-123
label_confidence	0.92

Data Leakage Rule

Training/testing split should be done by well, not by individual rows.

Recommended:

- GroupKFold
- Leave-one-well-out
- Field-level holdout where data permits

14. Model Training Pipeline

```
INGEST
↓
CLEAN
↓
LABEL
↓
DEPTH WINDOWING
↓
FEATURE ENGINEERING
↓
GROUPED TRAIN/VALIDATION/TEST SPLIT
↓
TRAIN
↓
CALIBRATE
↓
EVALUATE
↓
MODEL REGISTRY
↓
DEPLOY
↓
MONITOR
↓
RETRAIN
```

Evaluation

Classification:

- Precision
- Recall
- F1
- PR-AUC / ROC-AUC where appropriate

Probability:

- Brier score
- Calibration curve

Retrieval:

- Recall@K
- NDCG

RAG:

- Retrieval recall
- Evidence correctness
- Citation coverage

15. Risk Engine

A prototype risk engine can combine:

- ML probability
- Historical event frequency
- Similarity-weighted event frequency
- Geological risk features
- Live anomaly score

Conceptually:

```
RiskScore =  
  w1 * MLProbability  
+ w2 * HistoricalFrequency  
+ w3 * SimilarityWeightedFrequency  
+ w4 * GeologicalRisk  
+ w5 * LiveAnomalyScore
```

The weights must be validated using held-out data.

If sufficient data does not exist to justify calibrated probability, show:

- historical event counts
- evidence strength
- qualitative risk bands

instead of unsupported precise percentages.

16. Depth-Wise Risk Profile

Example:

Depth	Historical Pattern	Output
0–1000 m	Little comparable evidence	Low / informational
1000–2000 m	Rare events	Low
2000–2500 m	Repeated comparable events	Moderate
2500–3000 m	Strong offset-well evidence	High / review

Each result should include:

- Evidence wells
- Event count
- Confidence
- Data quality
- Model version

- Supporting reports

17. RAG / AI Assistant Pipeline

```
USER QUESTION
↓
INTENT EXTRACTION
↓
STRUCTURED FILTERS
↓
SQL / POSTGIS SEARCH
+
VECTOR SEARCH
↓
RERANKING
↓
EVIDENCE PACKAGE
↓
LLM
↓
SOURCE-CITED ANSWER
```

Example:

Show nearby wells that experienced mud loss around 2800 m.

Backend interpretation:

```
event_type = mud_loss
target_depth ≈ 2800 m
radius = user-selected radius
```

Structured events are retrieved first.

Relevant report passages are then retrieved as supporting evidence.

The LLM summarizes only the retrieved evidence.

18. API Specification

Method	Endpoint	Purpose
POST	/api/v1/well-plans	Create proposed well plan
GET	/api/v1/wells/nearby	Nearby wells
GET	/api/v1/wells/{id}	Well profile
GET	/api/v1/wells/{id}/events	Historical events
POST	/api/v1/search/events	Event search
POST	/api/v1/risk/predict	Generate risk
GET	/api/v1/risk/{plan_id}	Retrieve risk
POST	/api/v1/recommendations	Recommendations
POST	/api/v1/assistant/query	AI assistant
POST	/api/v1/ingestion/jobs	Start ingestion
GET	/api/v1/ingestion/jobs/{id}	Job status
WS	/ws/live/{well_id}	Live updates
GET	/api/v1/alerts	Active alerts

19. Real-Time eRTMAC / WITSML Pipeline

For SIH, historical WITSML/DDR data can be replayed as simulated live data.

Production mode uses an authorized live source.

```
LIVE STREAM
↓
INGEST
↓
BUFFER
↓
VALIDATE
↓
RESAMPLE
↓
FEATURE WINDOW
↓
ONLINE MODEL
↓
RULE CHECK
↓
ALERT SERVICE
↓
WEBSOCKET
↓
DASHBOARD
```

Alert Record

Every alert should contain:

- Timestamp
- Depth
- Feature snapshot
- Model version
- Rule version
- Evidence
- Status
- Acknowledgement history

Alert cooldown and deduplication should be implemented.

20. Recommended Technology Stack

Area	Technology
Frontend	React / Next.js + Tailwind
Maps	Leaflet / MapLibre / Mapbox
Backend	Python FastAPI
Database	PostgreSQL + PostGIS + pgvector
Storage	S3-compatible storage / MinIO
Async Jobs	Redis + Celery/RQ or FastAPI workers
PDF / HTML	PyMuPDF, pdfplumber, BeautifulSoup, lxml
OCR	Tesseract / PaddleOCR
Well Logs	lasio + WITSML/XML parser
ML	scikit-learn + XGBoost / LightGBM
RAG	sentence-transformers + pgvector/Qdrant + approved LLM
Monitoring	Prometheus + Grafana
Deployment	Docker Compose for SIH; on-prem/Kubernetes later
Auth	JWT/OIDC + RBAC

21. Suggested Backend Repository

```
backend/  
  app/  
    api/  
      routes_wells.py  
      routes_events.py  
      routes_risk.py  
      routes_assistant.py  
      routes_ingestion.py  
      routes_alerts.py  
  
    core/  
      config.py  
      security.py  
  
    db/  
      models/  
      repositories/  
  
    services/  
      spatial_service.py  
      document_service.py  
      event_service.py  
      risk_service.py  
      recommendation_service.py  
      rag_service.py  
      alert_service.py  
  
    ingestion/  
      pdf/  
      las/  
      witsml/  
      csv/  
  
    ml/  
      features/  
      training/  
      inference/  
      evaluation/  
  
    workers/  
      jobs.py  
  
  tests/  
  
  data/  
    raw/  
    processed/  
    features/  
  
  models/  
    registry/
```

22. Security and Governance

Required controls:

- RBAC
- Authentication
- Encryption in transit
- Encryption at rest
- Audit logging
- Source-level RAG permissions
- Immutable source document storage
- Model/version tracking
- Secure API access

Confidential enterprise data should not be sent to external AI services without explicit authorization.

The prototype must not autonomously control drilling equipment.

23. SIH MVP

Phase 1

Load Volve + selected NLOG data and normalize wells, logs and formations.

Phase 2

Interactive map + radius search + well profiles.

Phase 3

Historical event extraction + depth-wise event visualization.

Phase 4

Well similarity + evidence-backed risk profile.

Phase 5

RAG assistant with citations.

Phase 6

WITSML/DDR replay as simulated live monitoring and alerts.

Phase 7

Final dashboard + evaluation report + demo workflow.

24. Technical Risks and Mitigations

Risk	Mitigation
No exact public Oil India dataset	Use Volve/NLOG for prototype and keep adapters ready
Sparse event labels	Expert annotation + weak supervision + confidence flags
Data leakage	Well/field-level holdouts
Different geology	Geology-aware similarity + domain validation
LLM hallucination	Retrieval-grounded answers + citations
False alerts	Calibration + thresholds + cooldown + human review
Safety misuse	Decision-support only; no autonomous control

25. Transition to Oil India Production

When authorized Oil India data becomes available:

1. Add Oil India-specific connectors.
2. Map internal field and formation nomenclature.
3. Import historical WCR/DDR data.
4. Refine event taxonomy with domain experts.
5. Retrain models.
6. Calibrate probabilities.
7. Validate with drilling/geology SMEs.
8. Run shadow-mode alerts.
9. Move to controlled operational deployment after validation.

The existing architecture remains reusable.

26. Final Technical Definition

NWIS is a modular platform that builds an institutional-memory layer from historical well reports and structured data, discovers nearby wells, aligns geology and depth information, identifies historical drilling events, estimates evidence-supported risk where data quality allows, surfaces historical mitigation practices, and provides a path to live early-warning analytics using authorized drilling streams.

The core engineering principle is:

Every important output should be traceable to a source record, model version, confidence value and evidence trail.

27. Open Data References

- Equinor Volve Open Data: <https://www.equinor.com/energy/volve-data-sharing>
- NLOG Data: <https://www.nlog.nl/en/data>
- NLOG Boreholes: <https://www.nlog.nl/en/boreholes>
- NLOG PSNS: <https://www.nlog.nl/en/pressure-southern-north-sea-psns-database>
- NLOG Production and Injection Data: <https://www.nlog.nl/en/production-and-injection-data>